Advanced Rust (2026): Unsafe Rust

Lecture 5

Lukáš Hozda

Spring 2026

MFF CUNI

Unsafe Rust

Unsafe Rust

- Rust effectively contains two closely related languages: safe Rust and unsafe Rust

Unsafe Rust

- Rust effectively contains two closely related languages: safe Rust and unsafe Rust
- `unsafe` is a boundary where the compiler stops proving some properties for you

Unsafe Rust

- Rust effectively contains two closely related languages: safe Rust and unsafe Rust
- `unsafe` is a boundary where the compiler stops proving some properties for you
- It is neither “just C” nor something to fear superstitiously

Unsafe Rust

- Rust effectively contains two closely related languages: safe Rust and unsafe Rust
- `unsafe` is a boundary where the compiler stops proving some properties for you
- It is neither “just C” nor something to fear superstitiously
- It is a tool for cases the compiler cannot verify on its own

What Safe Rust Gives Us

- No dangling reference dereference

What Safe Rust Gives Us

- No dangling reference dereference
- No out-of-bounds access through safe indexing

What Safe Rust Gives Us

- No dangling reference dereference
- No out-of-bounds access through safe indexing
- No access to uninitialized values through safe APIs

What Safe Rust Gives Us

- No dangling reference dereference
- No out-of-bounds access through safe indexing
- No access to uninitialized values through safe APIs
- No data races in threaded safe code

What Safe Rust Gives Us

- No dangling reference dereference
- No out-of-bounds access through safe indexing
- No access to uninitialized values through safe APIs
- No data races in threaded safe code
- No violation of reference aliasing rules through safe code

What `unsafe` Permits

- Dereferencing raw pointers

What `unsafe` Permits

- Dereferencing raw pointers
- Calling unsafe functions

What `unsafe` Permits

- Dereferencing raw pointers
- Calling unsafe functions
- Accessing or modifying mutable statics

What `unsafe` Permits

- Dereferencing raw pointers
- Calling unsafe functions
- Accessing or modifying mutable statics
- Implementing unsafe traits

What `unsafe` Permits

- Dereferencing raw pointers
- Calling unsafe functions
- Accessing or modifying mutable statics
- Implementing unsafe traits
- Accessing union fields

Why Do We Have Unsafe

- Some correct programs cannot be proven safe by the compiler

Why Do We Have Unsafe

- Some correct programs cannot be proven safe by the compiler
- Some tasks require direct contact with hardware or foreign code

Why Do We Have Unsafe

- Some correct programs cannot be proven safe by the compiler
- Some tasks require direct contact with hardware or foreign code
- Concurrency primitives and low-level collections need capabilities safe Rust does not expose

Why Do We Have Unsafe

- Some correct programs cannot be proven safe by the compiler
- Some tasks require direct contact with hardware or foreign code
- Concurrency primitives and low-level collections need capabilities safe Rust does not expose
- Sometimes unsafe is used for performance, though that is the weakest reason unless measured

Where Unsafe Appears

```rust
const DEVICE_REGISTER_ADDRESS: *mut u32 = 0x1000_0000 as *mut u32;

unsafe fn write_to_device_register(value: u32) {
    std::ptr::write_volatile(DEVICE_REGISTER_ADDRESS, value);
}

fn main() {
    unsafe {
        write_to_device_register(0xABCD_EF01);
    }
}
```

Hardware Notes

- Compiler cannot know whether that address is valid

Hardware Notes

- Compiler cannot know whether that address is valid
- Compiler cannot know whether concurrent access is safe

Hardware Notes

- Compiler cannot know whether that address is valid
- Compiler cannot know whether concurrent access is safe
- On ordinary hosted systems, this code is usually nonsense

Hardware Notes

- Compiler cannot know whether that address is valid
- Compiler cannot know whether concurrent access is safe
- On ordinary hosted systems, this code is usually nonsense
- On bare metal, this may be exactly the right thing to do

Another Hardware Example

```
const VGA_PTR: *mut u8 = 0x8080_8080 as *mut u8;

let vga = unsafe {
    std::slice::from_raw_parts_mut(VGA_PTR, 800 * 600)
};
```

- Only safe under external conditions about the machine and memory map

Another Hardware Example

```
const VGA_PTR: *mut u8 = 0x8080_8080 as *mut u8;

let vga = unsafe {
    std::slice::from_raw_parts_mut(VGA_PTR, 800 * 600)
};
```

- Only safe under external conditions about the machine and memory map
- HALs hide this for ordinary users, but the HAL itself must be written somehow

Foreign Functions

```
extern "C" {
    fn strcpy(
        dest: *mut libc::c_char,
        src: *const libc::c_char,
    ) -> *mut libc::c_char;
}
```

- Rust cannot inspect what the foreign function actually does

Safe Wrapper Around FFI

```
fn safe_strcpy(dest: &mut [u8], src: &str) -> Result<(), &'static str> {
    let src_len = src.len();
    if dest.len() <= src_len {
        return Err("destination buffer is too small");
    }
```

```
    let c_src = std::ffi::CString::new(src).expect("CString::new failed");
    let src_ptr = c_src.as_ptr();
    let dest_ptr = dest.as_mut_ptr() as *mut libc::c_char;

    unsafe { strcpy(dest_ptr, src_ptr); }
    Ok(())
}
```

Concurrency Primitives

- `Mutex<T>` can manufacture `&mut T` from `&self`

Concurrency Primitives

- `Mutex<T>` can manufacture `&mut T` from `&self`
- The compiler alone cannot prove that this does not create aliasing violations

Concurrency Primitives

- `Mutex<T>` can manufacture `&mut T` from `&self`
- The compiler alone cannot prove that this does not create aliasing violations
- Unsafe code is required to bridge from raw state management to safe API

Raw Memory and Reference Tricks

Transmuting References

```
use std::mem;

struct BufferHolder {
    buffer: *mut i32,
    len: usize,
}
```

```rust
impl BufferHolder {
    fn new(size: usize) -> Self {
        let mut vec = vec![0; size];
        let buffer_ptr = vec.as_mut_ptr();
        mem::forget(vec);
        BufferHolder { buffer: buffer_ptr, len: size }
    }
```

```
    unsafe fn get_buffer_mut<'a>(&'a self) -> &'a mut [i32] {
        assert!(!self.buffer.is_null(), "buffer is null");
        std::mem::transmute::<(*mut i32, usize), &'a mut [i32]>((self.buffer,
self.len))
    }
}
```

Manual Drop Repair

```
impl Drop for BufferHolder {
    fn drop(&mut self) {
        unsafe {
            let _vec = Vec::from_raw_parts(self.buffer, self.len, self.len);
        }
    }
}
```

- Once we escape ordinary ownership, we must reconstruct destruction manually

Manual Drop Repair

```
impl Drop for BufferHolder {
    fn drop(&mut self) {
        unsafe {
            let _vec = Vec::from_raw_parts(self.buffer, self.len, self.len);
        }
    }
}
```

- Once we escape ordinary ownership, we must reconstruct destruction manually
- This is exactly the terrain where double-free and leak bugs are born

Memory Leak Is Not UB

```rust
use std::mem;

pub fn strify(s: String) -> &'static str {
    let s_ref = s.as_str();
    let s_static_ref: &'static str = unsafe { mem::transmute(s_ref) };
    mem::forget(s);
    s_static_ref
}
```

- This leaks memory, but leaking memory is not itself undefined behavior

Memory Leak Is Not UB

```rust
use std::mem;

pub fn strify(s: String) -> &'static str {
    let s_ref = s.as_str();
    let s_static_ref: &'static str = unsafe { mem::transmute(s_ref) };
    mem::forget(s);
    s_static_ref
}
```

- This leaks memory, but leaking memory is not itself undefined behavior
- Safe Rust also allows leaks through `Rc` cycles and deliberate retention

Borrow-Checker Limitation Example

```
fn next(buffer: &mut String) -> &str {
    loop {
        let event = parse(buffer);
        if true {
            return event;
        }
    }
}

fn parse(buffer: &mut String) -> &str { unimplemented!() }
```

- There are correct programs the borrow checker cannot yet express or accept

Performance Example

```
fn sum_vector_safe(v: &[i32]) -> i32 {
    let mut sum = 0;
    for &item in v.iter() {
        sum += item;
    }
    sum
}
```

```rust
fn sum_vector_unsafe(v: &[i32]) -> i32 {
    let mut sum = 0;
    let len = v.len();
    unsafe {
        let ptr = v.as_ptr();
        for i in 0..len {
            sum += *ptr.add(i);
        }
    }
    sum
}
```

- This is not an argument for using unsafe by default

```rust
fn sum_vector_unsafe(v: &[i32]) -> i32 {
    let mut sum = 0;
    let len = v.len();
    unsafe {
        let ptr = v.as_ptr();
        for i in 0..len {
            sum += *ptr.add(i);
        }
    }
    sum
}
```

- This is not an argument for using unsafe by default
- Measure first, then decide whether the complexity pays for itself

Undefined Behavior

Compiler Assumptions

- The compiler assumes all code, including unsafe code, respects Rust's safety rules

Compiler Assumptions

- The compiler assumes all code, including unsafe code, respects Rust's safety rules
- It assumes no dangling dereference, no invalid values, no unsynchronized write races, and no broken aliasing

Compiler Assumptions

- The compiler assumes all code, including unsafe code, respects Rust's safety rules
- It assumes no dangling dereference, no invalid values, no unsynchronized write races, and no broken aliasing
- Optimizations are built on these assumptions

UB Example

```
fn paradox() -> ! {
    unsafe { std::mem::transmute(()) }
}
```

- The never type has no values

UB Example

```
fn paradox() -> ! {
    unsafe { std::mem::transmute(()) }
}
```

- The never type has no values
- Producing one is undefined behavior

UB Example

```
fn paradox() -> ! {
    unsafe { std::mem::transmute(()) }
}
```

- The never type has no values
- Producing one is undefined behavior
- Undefined behavior is not an exception mechanism, it is loss of semantic footing

Aliasing Example

```
fn compute(i: &u32, o: &mut u32) {
    if *i > 10 {
        *o = 1;
    }

    if *i > 5 {
        *o *= 2;
    }
}
```

```rust
fn compute(i: &u32, o: &mut u32) {
    let cached_i = *i;
    if cached_i > 10 {
        *o = 2;
    } else if cached_i > 5 {
        *o *= 2;
    }
}
```

- The optimizer expects reference aliasing rules to hold

```rust
fn compute(i: &u32, o: &mut u32) {
    let cached_i = *i;
    if cached_i > 10 {
        *o = 2;
    } else if cached_i > 5 {
        *o *= 2;
    }
}
```

- The optimizer expects reference aliasing rules to hold
- If unsafe code constructs invalid aliasing, transformations like this stop being justified

Panic Safety in Unsafe Code

```
impl<T> Vec<T> {
    fn extend_map<U, F>(&mut self, us: &[U], mut f: F)
    where
        F: FnMut(&U) -> T,
    {
        self.reserve(us.len());
        let cur_len = self.len();
        unsafe { self.set_len(cur_len + us.len()); }
```

```
        let mut into = unsafe { self.as_mut_ptr().add(cur_len) };
        for u in us {
            unsafe {
                std::ptr::write(into, f(u));
                into = into.add(1);
            }
        }
    }
}
```

Why This Is Subtle

- What if `f(u)` panics halfway through

Why This Is Subtle

- What if `f(u)` panics halfway through
- `set_len` already told `Vec` that the new elements exist

Why This Is Subtle

- What if `f(u)` panics halfway through
- `set_len` already told `Vec` that the new elements exist
- During unwinding, drop may try to drop uninitialized elements

Why This Is Subtle

- What if `f(u)` panics halfway through
- `set_len` already told `Vec` that the new elements exist
- During unwinding, drop may try to drop uninitialized elements
- Unsafe wrappers must remain correct even under panic

Safe Wrapper Obligation

- A safe interface around unsafe internals must behave correctly for all safe callers

Safe Wrapper Obligation

- A safe interface around unsafe internals must behave correctly for all safe callers
- It is not enough that the wrapper works for intended use

Safe Wrapper Obligation

- A safe interface around unsafe internals must behave correctly for all safe callers
- It is not enough that the wrapper works for intended use
- It must survive weird but legal caller behavior too

Tools

Miri

- Miri is an interpreter for Rust's mid-level IR

Miri

- Miri is an interpreter for Rust's mid-level IR
- It can detect many classes of UB-related issues during execution

Miri

- Miri is an interpreter for Rust's mid-level IR
- It can detect many classes of UB-related issues during execution
- It is not a proof system, but it is extremely useful for unsafe code exploration

Running Miri

```
rustup +nightly component add miri
cargo +nightly miri test
```

Miri Example

```
#![feature(vec_into_raw_parts)]

struct Hehe(Vec<u8>);

impl Hehe {
    fn new() -> Self {
        let v = Vec::with_capacity(10);
        let (parts_ptr, _, parts_cap) = v.into_raw_parts();
```

```
        let v = unsafe { Vec::from_raw_parts(parts_ptr, 10, parts_cap) };
        Self(v)
    }
}

fn main() {
    let data = Hehe::new();
    println!("Uninitialized data: {}", data.0[0]);
}
```

- This reads uninitialized memory

```
        let v = unsafe { Vec::from_raw_parts(parts_ptr, 10, parts_cap) };
        Self(v)
    }
}

fn main() {
    let data = Hehe::new();
    println!("Uninitialized data: {}", data.0[0]);
}
```

- This reads uninitialized memory
- Miri is very good at catching this kind of mistake

Summary

- Unsafe exists for real reasons, but it trades compiler proof for manual proof

Summary

- Unsafe exists for real reasons, but it trades compiler proof for manual proof
- UB matters because the optimizer assumes it never happens

Summary

- Unsafe exists for real reasons, but it trades compiler proof for manual proof
- UB matters because the optimizer assumes it never happens
- Safe wrappers around unsafe code must be correct under all safe caller behavior

Summary

- Unsafe exists for real reasons, but it trades compiler proof for manual proof
- UB matters because the optimizer assumes it never happens
- Safe wrappers around unsafe code must be correct under all safe caller behavior
- Keep unsafe small, explicit, and justified

Questions?